

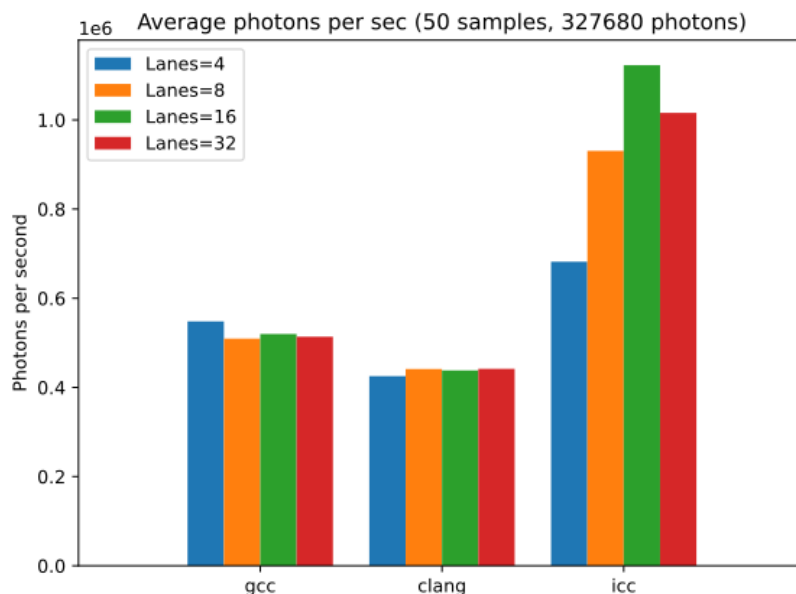
Computación Paralela- Laboratorio 3 - OpenMP

Computación paralela 2021-FAMAF

Cremona Javier
Castillo Marcelo

Resultados del Laboratorio 2 en JupiterAce

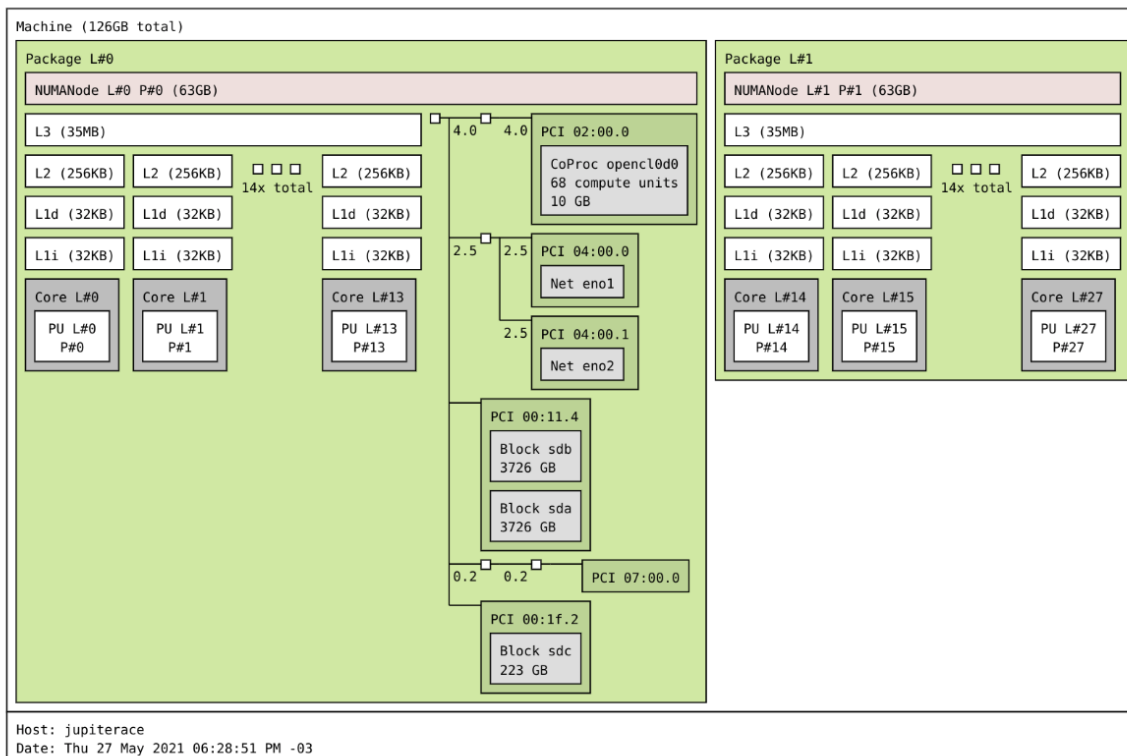
Para el laboratorio 2, el mejor resultado obtenido fue de 1.1 M photons per second (pps), usando el compilador icc, para 327680 fotones simulados y 16 “lanes” (tamaño de los arreglos utilizados para reemplazar las variables escalares).



Laboratorio 3

El objetivo del presente trabajo es usar OpenMP para sacar provecho de los múltiples cores de JupiterAce. Mediante OpenMP se busca repartir el trabajo entre los distintos cores para procesar fotones en paralelo.

JupiterAce cuenta con 28 cores en total y está conformado por dos nodos NUMA. Idealmente, nos gustaría que nuestro código con OpenMP corra 28 veces más rápido que la versión secuencial al utilizar 28 cores. Sabemos que en la práctica, hay limitaciones. Si bien un core puede ver a la memoria como un todo, acceder a la memoria local de su nodo NUMA es más barato que acceder a la memoria de otro nodo, y esto tiene implicancias en la performance. Otro limitante puede venir por el problema conocido como *False Sharing*. Este problema ocurre cuando distintos cores están trabajando sobre datos independientes, pero que comparten una misma línea de caché. Si un core modifica dicha línea de caché, esta es invalidada en el resto de los cores. Esto los obliga a releer los datos constantemente desde la RAM si están continuamente haciendo modificaciones. Nótese que *False Sharing* no afecta a la correctitud del programa (como podría hacerlo un race condition), solo la performance.



Modificaciones al código

Para usar OpenMP en nuestro código, simplemente llamé a `#pragma omp parallel` a nivel del main, y dentro del mismo se creó una instancia del generador de números aleatorios (PRNG), se crearon arrays (que resultan privados) para los histogramas y se llamó al método `photon()`. Esto implica que esto se ejecutará N veces, en paralelo en distintos hilos. El método `photon()` tiene la misma implementación que en el Laboratorio 2 (versión rand-ondemand, la que mejor resultados nos dio) salvo que ahora recibe el rng y los arrays para el histograma por parámetro (anteriormente eran globales). Adicionalmente hubo que indicar cuáles son los loops vectorizados con `#pragma omp simd`.

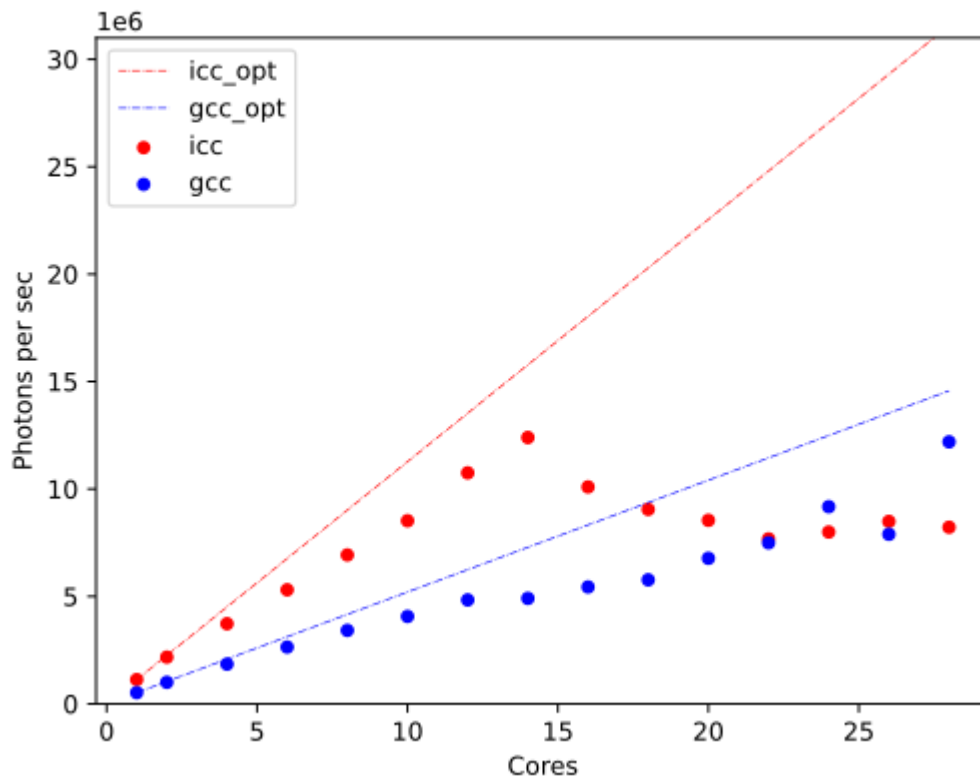
El método `photon()` guarda el calor en los arrays privados. Luego, al finalizar la ejecución de `photon()`, en el main se suman todos los arrays privados en un único histograma.

Resultados

Inicialmente se probó en JupiterAce con 327680 fotones simulados y 16 “lanes”. Se corrió `tiny_mc` para distinta cantidad de cores como puede verse (25 veces para cada cantidad de cores). Posteriormente, se aumentó x10 y x100 la cantidad de fotones simulados. En los siguientes resultados, los puntos representan la cantidad de pps obtenidos para cierta cantidad de cores empleados. La línea punteada indica la performance que se hubiese obtenido si emplear más cores escalara de forma perfecta.

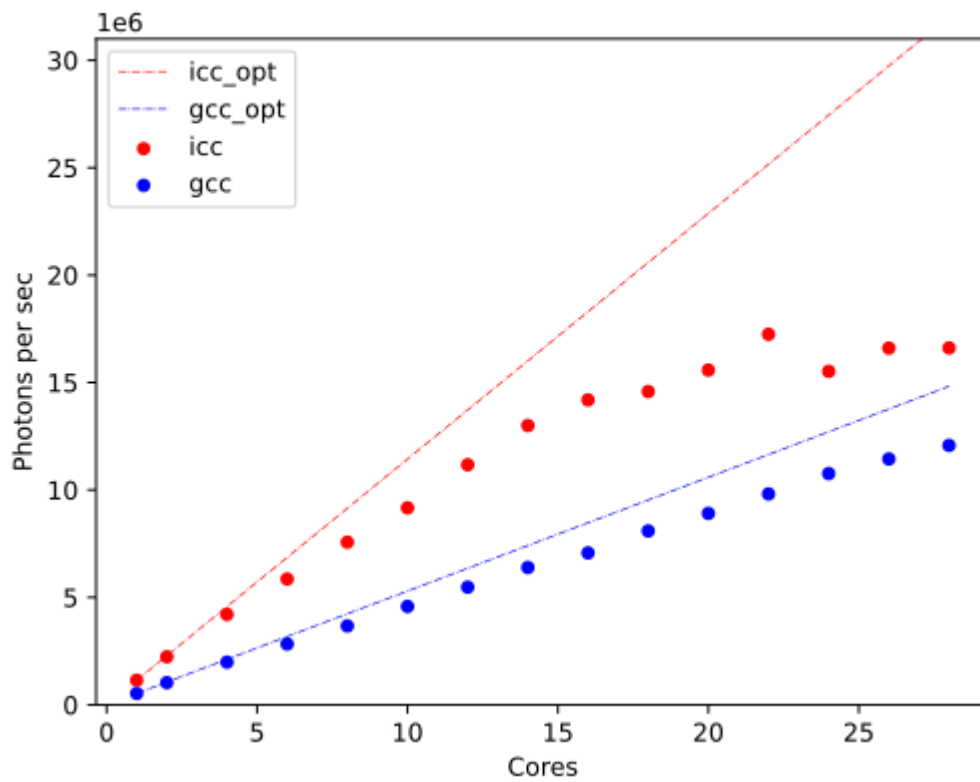
Caso 1:

Average photons per sec (25 samples, 327680 photons, 16 lanes)



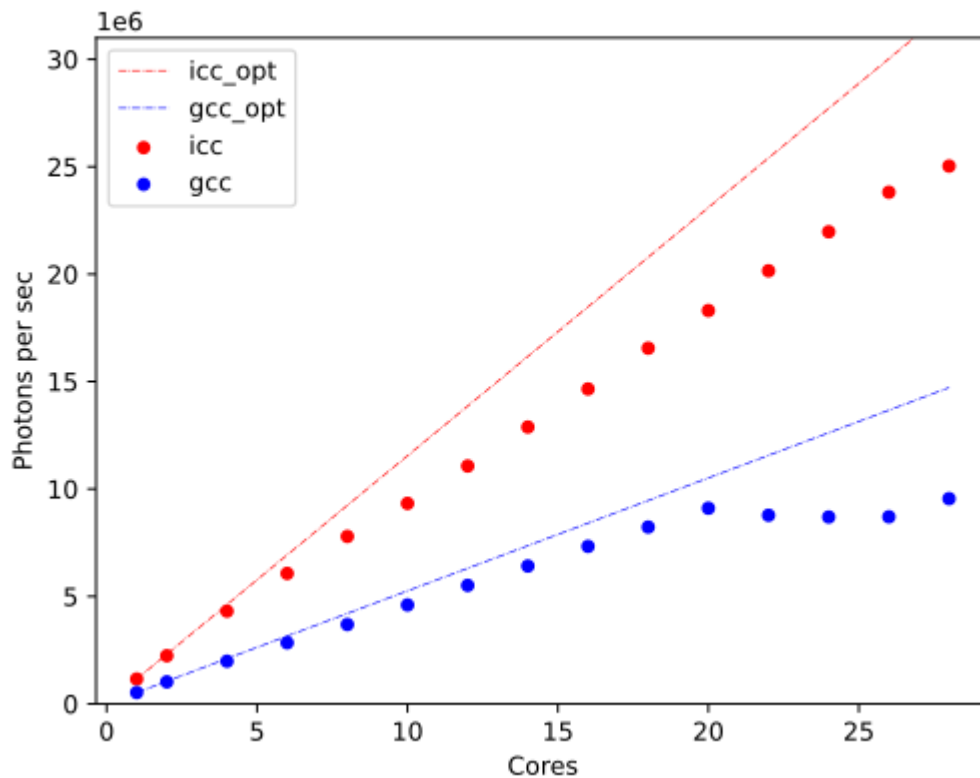
Caso 2:

Average photons per sec (25 samples, 3276800 photons, 16 lanes)



Caso 3:

Average photons per sec (25 samples, 32768000 photons, 16 lanes)



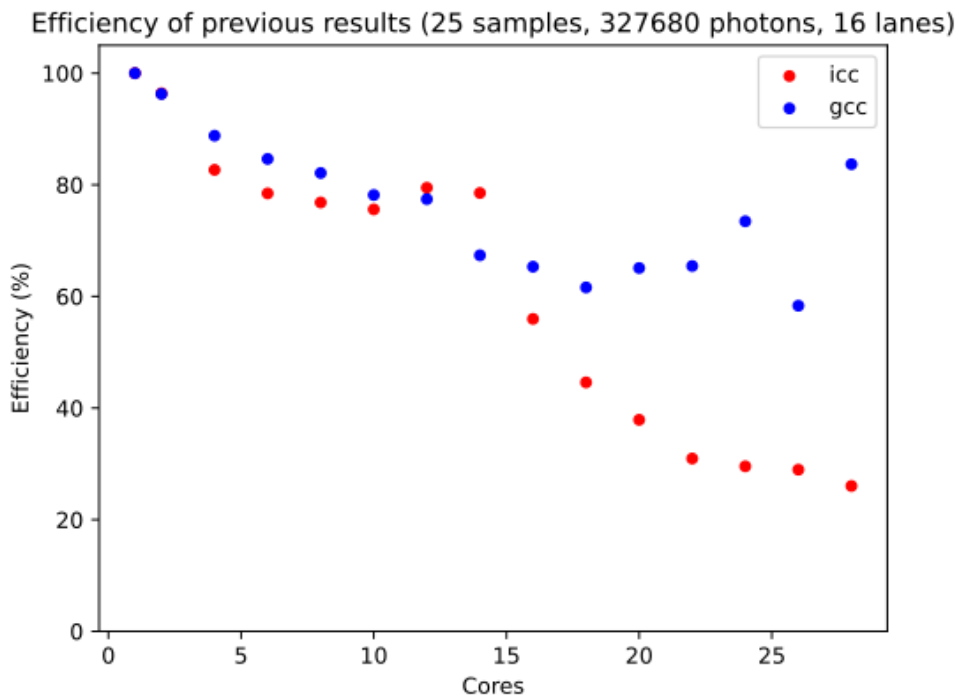
Para el caso 1, para el compilador icc vemos una caída a partir de los 14 cores. Esto podría indicar un potencial problema que tiene nuestro código en cuanto a los accesos de memoria. En un primer momento pensamos que podría tratarse de un problema al sumarizar los histogramas locales a cada hilo en el histograma global. Para ver si ese era el problema, comentamos las líneas en las que se sumaliza el histograma para que no se ejecuten. Luego corrimos tiny_mc nuevamente sobre las distintas cantidad de cores, pero nada cambió, seguíamos teniendo la caída a partir de los 14 cores. Para gcc, la caída empezaba a partir de los 12 cores pero no parecía tan marcada. Hay que decir que para 327680 fotones, los tiempos de simulación eran chicos, así que cualquier overhead nos impacta negativamente, pero no podía ser coincidencia que la caída se produzca justo a partir de los 14 cores.

Decidimos entonces aumentar la cantidad de fotones simulados por 10 (caso 2). Ahora la evolución parece comportarse mejor, pero de todas formas, la eficiencia es mala para muchos cores. Para gcc, a pesar de obtener menos pps, la escalabilidad parece ser un poco mejor.

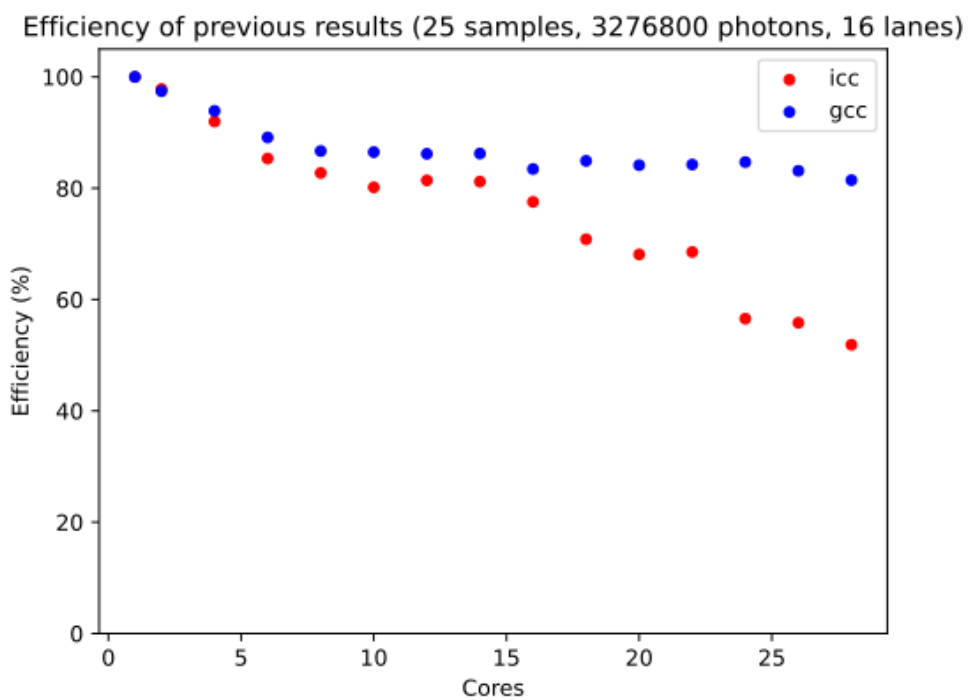
Por último, decidimos aumentar nuevamente la cantidad de fotones simulados por 10 (caso 3). De los 3 experimentos, aquí es donde mejor se aprovecha el uso de los 28 cores. Para icc se llega a un máximo de aproximadamente 25 M de pss, empleando los 28 cores. Para ver la eficiencia más claramente, realizamos nuevos gráficos para los 3 casos. Calculamos la eficiencia mediante la fórmula $E_n = P_n /$

($P_1 * n$), donde n es la cantidad de cores utilizados, P_n es la cantidad de pps obtenidos con la versión paralela de tiny_mc que usa n cores, y P_1 es la cantidad de pps obtenidos con la versión secuencial.

Caso 1:

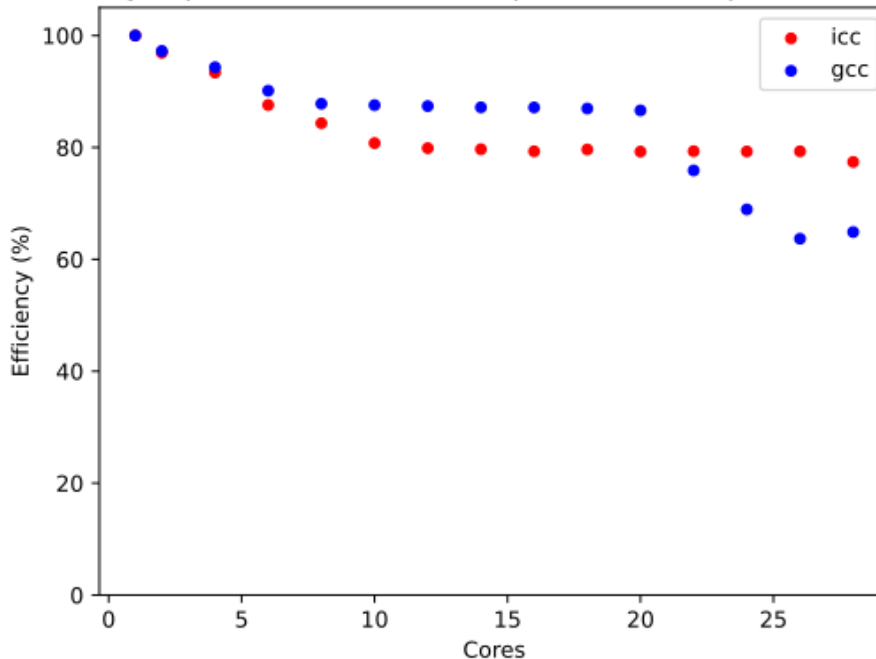


Caso 2:



Caso 3:

Efficiency of previous results (25 samples, 32768000 photons, 16 lanes)



Para más detalle puede verse la presentación: [Link a YouTube](#)

Allí puede verse también el análisis de Roofline. Para destacar es que parece que nuestro programa no le alcanza con la caché L1 y seguramente esto impacta en la performance. Esto puede deberse a los múltiples vectores de tamaño 16 que se emplearon para reemplazar las variables escalares en `photon()` y vectorizar el código.

Un problema a mencionar es que al emplear FDO con OpenMP, la primera corrida que junta las métricas para FDO demoraba mucho en ejecutar. Para icc, llegamos a ver que esa corrida era 150 veces más lenta que una ejecución ordinaria. Esto nos limitó a la hora de seguir probando con cantidades más grandes de fotones simulados. Aprovechando en este caso lo mucho que demoraba en correr `tiny_mc`, observamos la salida de `numatop`. Vimos que reportaba valores de ratio de RMA/LMA mayores a 1 en varios momentos, lo que significa que se están dando muchos accesos remotos a memoria, lo cual no debería ocurrir.

Otra cuestión es que, si bien corrimos `perf c2c` sobre `tiny_mc`, y nos reportó algunas líneas, no pudimos comprender la salida. Si bien en la sumariación del histograma podemos llegar a tener problemas de False Sharing, la realidad es que hacer padding en el arreglo global no nos mejoraba en nada.

Como conclusión, logramos que nuestro código vectorizado con icc, corra ahora 22 veces más rápido cuando se emplea OpenMP con 28 cores. Hay que mencionar

que para que se vea una mayor ganancia, hubo que aumentar la cantidad de fotones simulados en 2 órdenes de magnitud.